

## Praktikum 6

Thema: JavaDoc und innere Klassen

### Aufgabe 1: JavaDoc (30 Minuten)

In dieser Aufgabe dokumentieren Sie ein kleines Java-Programm mit **zwei Klassen** und erzeugen daraus eine HTML-Dokumentation mit JavaDoc.

Erstellen Sie ein neues Java-Projekt in Eclipse und importieren Sie die zwei Klassen **Calculator.java** und **CalculatorApp.java** aus OPAL in das Projekt („rein ziehen“).

Verstehen Sie das Programm und ergänzen Sie aussagekräftige JavaDoc-Kommentare für beide Klassen und alle Methoden. Achten Sie besonders darauf:

- Jede Methodenbeschreibung besteht aus mindestens zwei Sätzen.
- Der erste Satz ist eine kurze Zusammenfassung.
- Dokumentieren Sie außerdem Parameter (@param) und Rückgabewerte (@return, falls vorhanden).

Generieren Sie die JavaDoc-Dokumentation in Eclipse und schauen Sie sich die generierten HTML-Dokumente an, indem Sie die Datei index.html (mit Ihrem Browser) starten.

### Aufgabe 2: Innere Klassen (30 Minuten)

In dieser Aufgabe erweitern Sie die bestehende Klasse **Library.java** (aus OPAL) schrittweise durch eine **Mitgliedsklasse (innere Klasse)** und eine **lokale Klasse innerhalb einer Methode**.

#### a) Mitgliedsklasse erstellen

Erweitern Sie die Klasse Library um eine **innere Klasse Book** mit

- Attribute: `title (String)` und `author (String)`
- Konstruktor zur Initialisierung
- Methode `printBook()`, die Titel und Autor ausgibt

Ergänzen Sie in der Klasse `Library` eine Methode `public void addSampleBook()` und:

- Erzeugen Sie dort ein Objekt der inneren Klasse `Book`
- Geben Sie das Buch mit `printBook()` aus
- Rufen Sie die Methode in der `main`-Methode auf

### b) Lokale Klasse erstellen

Erweitern Sie die Methode `printLibrary()` um eine **lokale Klasse `Address`** mit dem Attribut `city(String)` und der Methode `printAddress()`.

Innerhalb der Methode `printLibrary()` sollen Sie außerdem ein Objekt der lokalen Klasse erzeugen und die Adresse ausgeben

## Aufgabe 3: Interface Comparator (30 Minuten)

**Ergänzende Erklärung vorab:** Im letzten Praktikum haben wir das Interface `Comparable` verwendet zum Sortieren. Objekte einer Klasse, die das Interface `Comparable` und damit die Methode `compareTo` implementieren, können sich selbst untereinander vergleichen, so dass das Sortieren ermöglicht wird. Diese Methode taugt aber nur, wenn die Objekte immer nur nach einem Kriterium sortiert werden. Möchte man Objekte nach verschiedenen Kriterien sortieren können – wie in dieser Aufgabe – brauchen wir das Interface `Comparator` mit der Methode `compare`. Damit bauen wir eine neue (hier anonyme) Klasse, die Objekte liefert, die zwei Objekte der ursprünglichen Klasse nach einem beliebigen Kriterium einordnen können. Wir können auch mehrere solcher `Comparator`-Objekte haben. Recherchieren Sie die offizielle JavaDoc-Dokumentation im Internet, wie die `compare`-Methode aussehen soll.

### Aufgabe:

In dieser Aufgabe arbeiten Sie mit einer Liste von **Produkten** und sortieren diese mithilfe eines Comparators, der als **anonyme innere Klasse** implementiert wird. Als Vorlage finden Sie die beiden Dateien **ShopApp.java** und **Product.java** im OPAL.

Die Klasse `Product` bleibt unverändert!

Sortieren Sie die Liste `products` zunächst nach dem **Preis (aufsteigend)**. Verwenden Sie zum Sortieren die Methode `Collections.sort(...)` und implementieren Sie den Comparator (als `Comparator<Product>`) als **anonyme innere Klasse** (dazu überschreiben Sie die Methode `compare(...)` → siehe Vorlesung!)

Sortieren Sie die Liste anschließend nach dem **Namen (alphabetisch)** und geben Sie diese ebenfalls aus.